

Abstract

Linux Inter-Process Communication Using Pipes and FIFOs is a system programming project that demonstrates how separate processes can communicate and exchange data in a Linux environment. Since processes normally have independent memory spaces, a suitable communication mechanism is required for them to share information and coordinate their activities. The main objective of this project is to understand and implement IPC using *unnamed pipes and named pipes (FIFOs)* while demonstrating process creation, communication, synchronization, and resource management. The proposed approach begins by creating processes using `fork()`. An unnamed pipe is then used to establish communication between related processes, such as a parent and child, where one process writes data and the other reads it. For communication between independent processes, a FIFO is created using `mkfifo()`, allowing processes to exchange data through a named communication channel. The project will be implemented in *C on Linux/Ubuntu* using Linux/POSIX system-programming interfaces. Important system calls and functions include *`fork()`, `pipe()`, `mkfifo()`, `open()`, `read()`, `write()`, `close()`, and `wait()`. Proper synchronization and error handling will be included to ensure reliable communication and safe resource usage. Different test cases will be performed by changing the number of processes and the messages exchanged between them. The expected outcome is a working Linux-based application that clearly demonstrates communication between related and independent processes. The project will provide practical understanding of **process creation, IPC, pipes, FIFOs, synchronization, system calls, and Linux/POSIX programming*, with results verified through terminal outputs and test cases.